

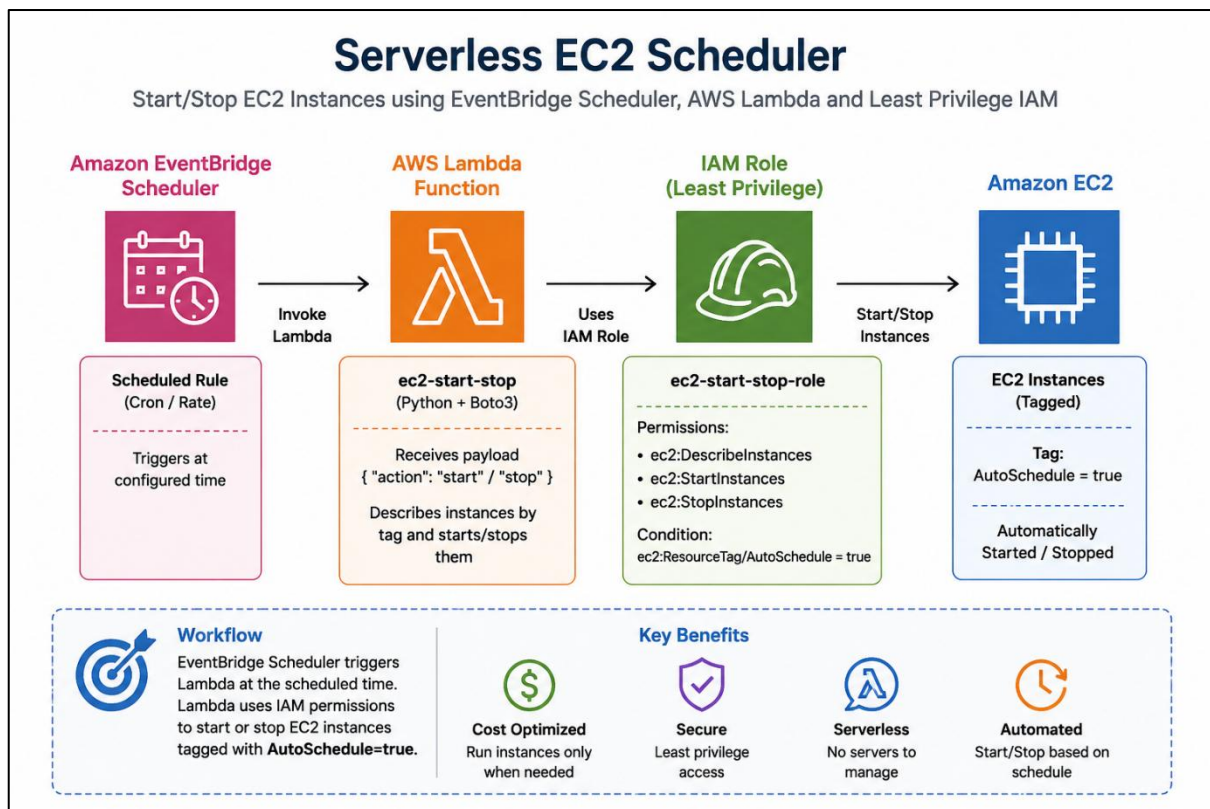
Serverless EC2 Instance Scheduler using AWS Lambda

Prepared By: Pooja Daingade

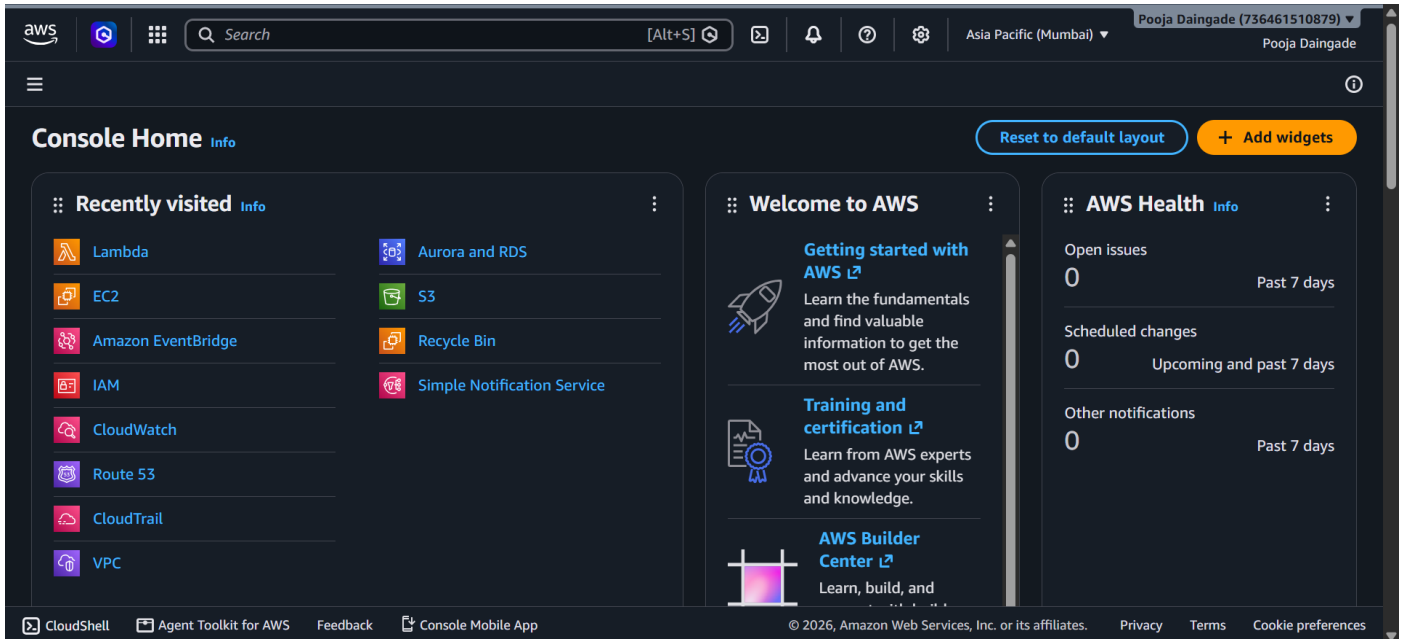
Project Overview

This project implements a serverless solution to automate the start and stop of Amazon EC2 instances using AWS Lambda. Amazon EventBridge is used to trigger the Lambda function on a scheduled basis, while AWS IAM provides least-privilege permissions required to manage the EC2 instance. The solution helps reduce manual effort, improve resource management, and optimize AWS costs by automatically controlling EC2 instance usage.

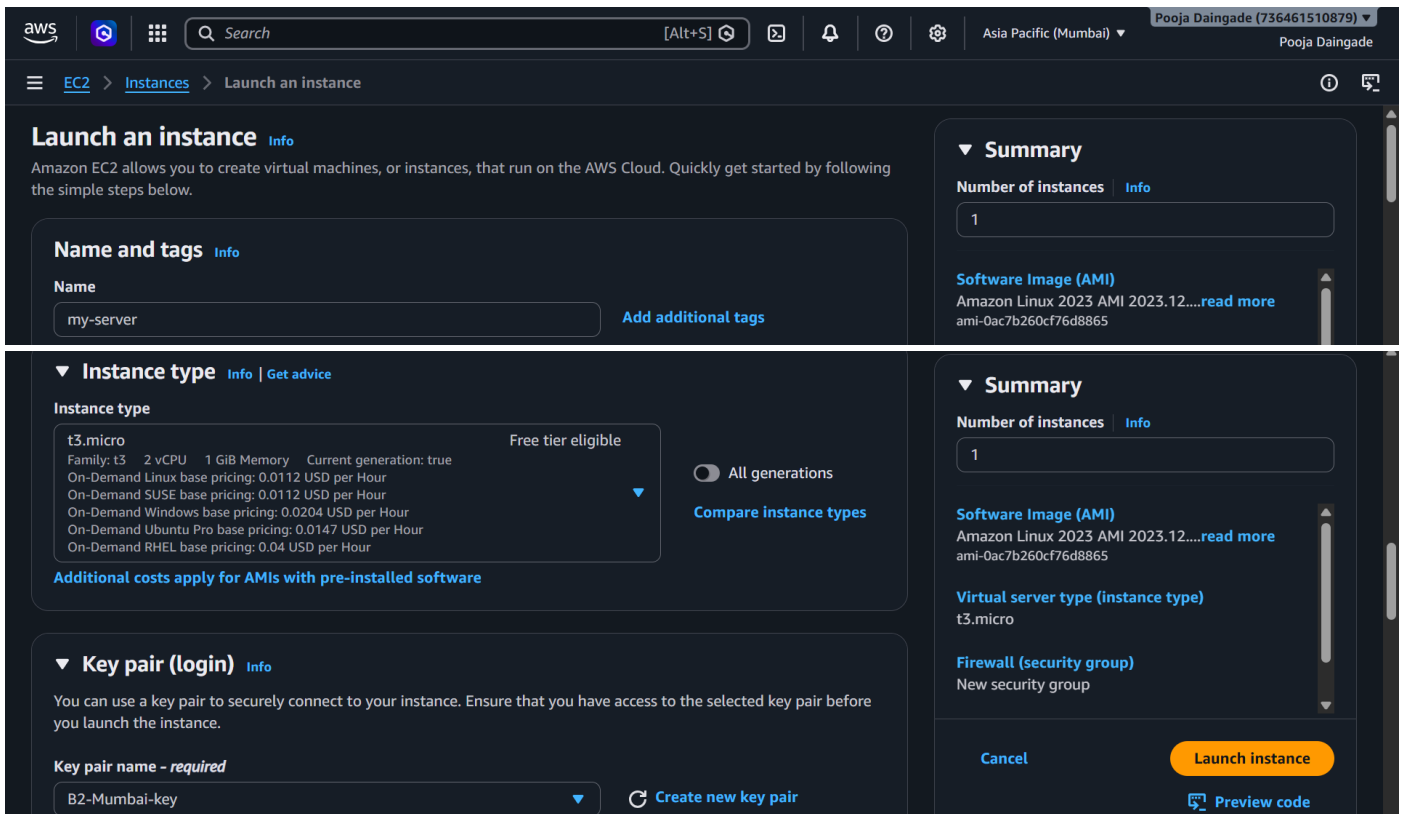
Architecture Diagram



1. Logged in to the AWS Management Console and accessed the required AWS services for the project, including EC2, Lambda, IAM, and EventBridge.



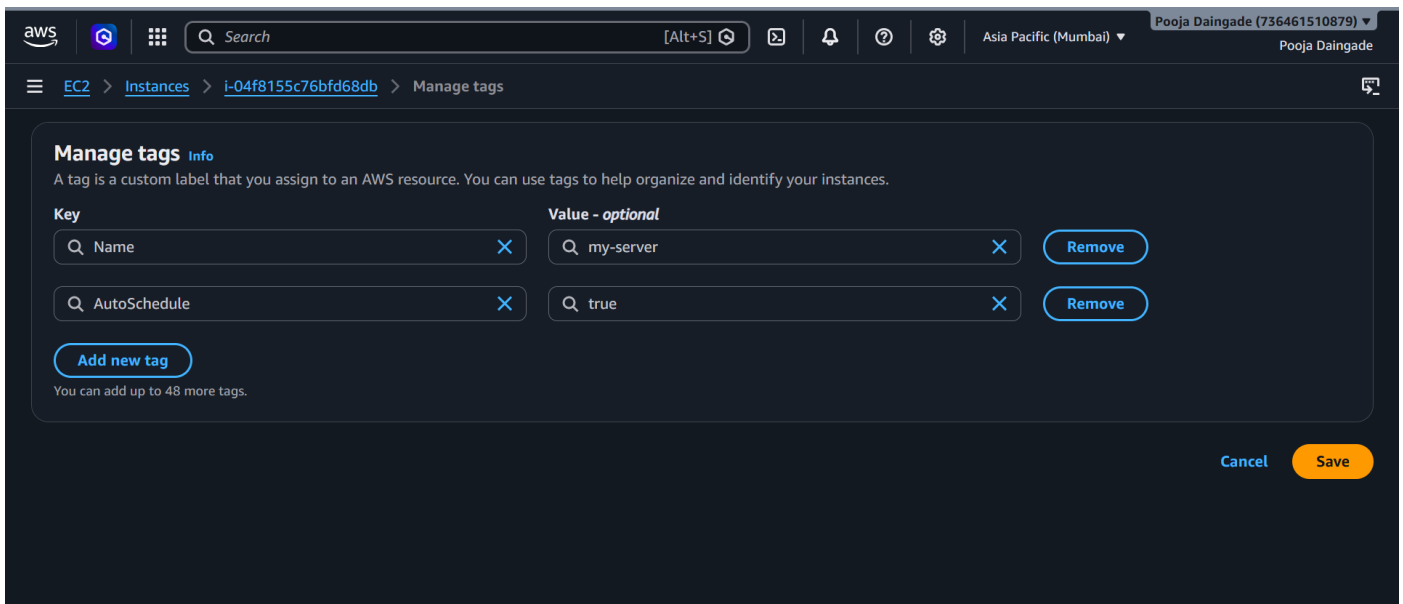
2. Created the EC2 instance by selecting Amazon Linux 2023 as the AMI, t3.micro as the instance type, and the required key pair for secure access.



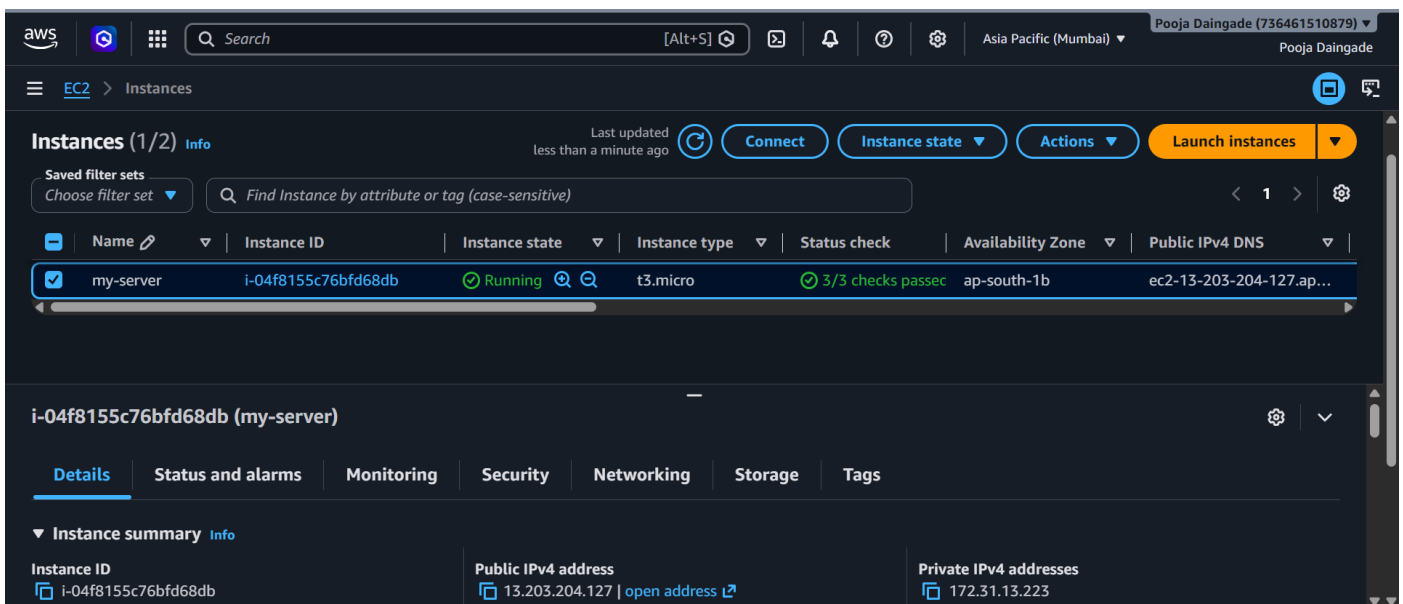
3. Adding Tags to the EC2 Instance

Key = **AutoSchedule**

Value = **true**



4. Successfully launched the EC2 instance **my-server** and verified that it is in the Running state with all status checks passed.

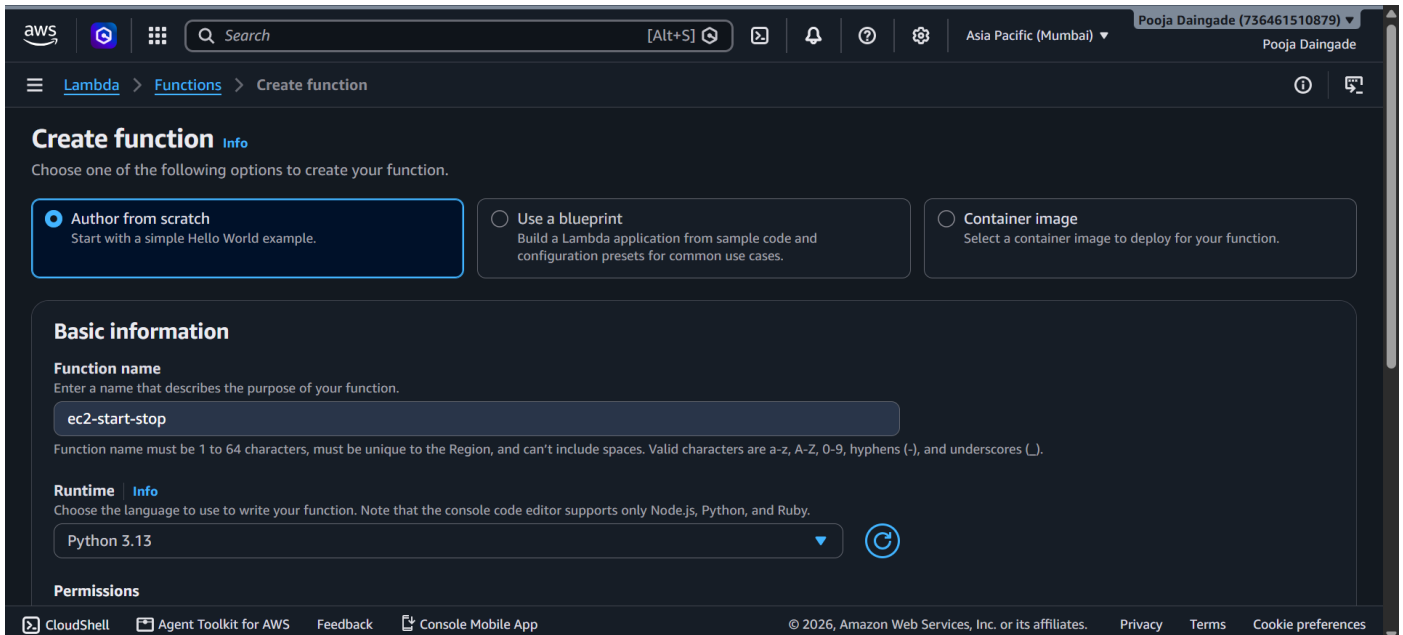


5. Creating the AWS Lambda Function: Selected Author from scratch

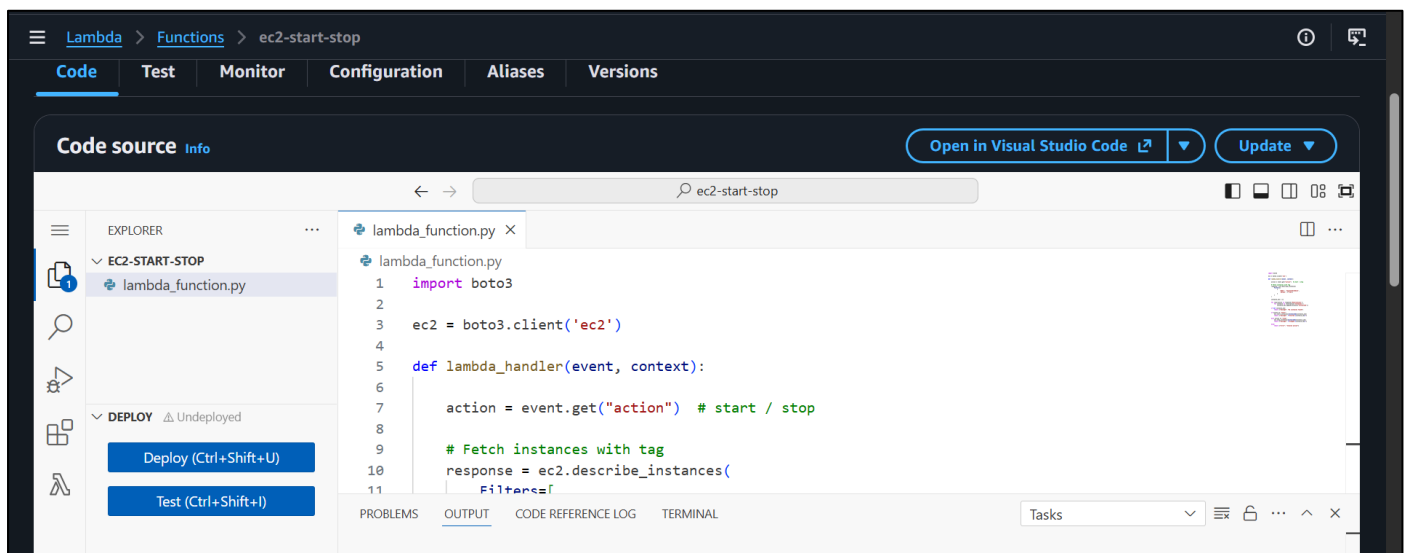
Function name: **ec2-start-stop**

Runtime: **Python 3.13**

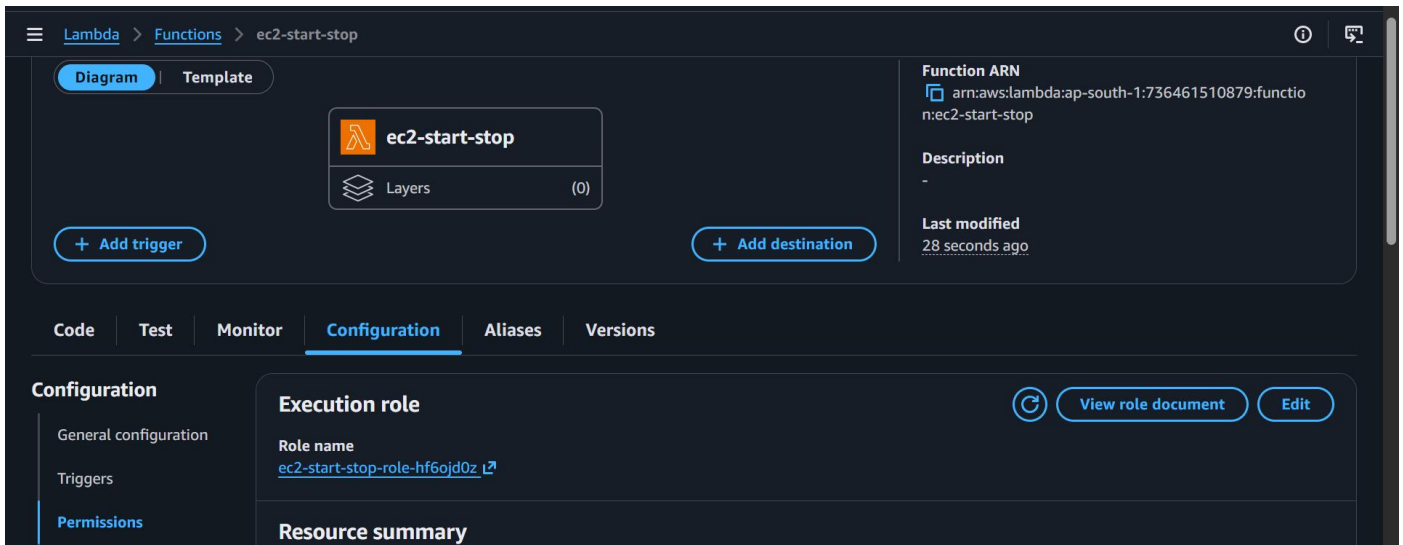
Click: Create function



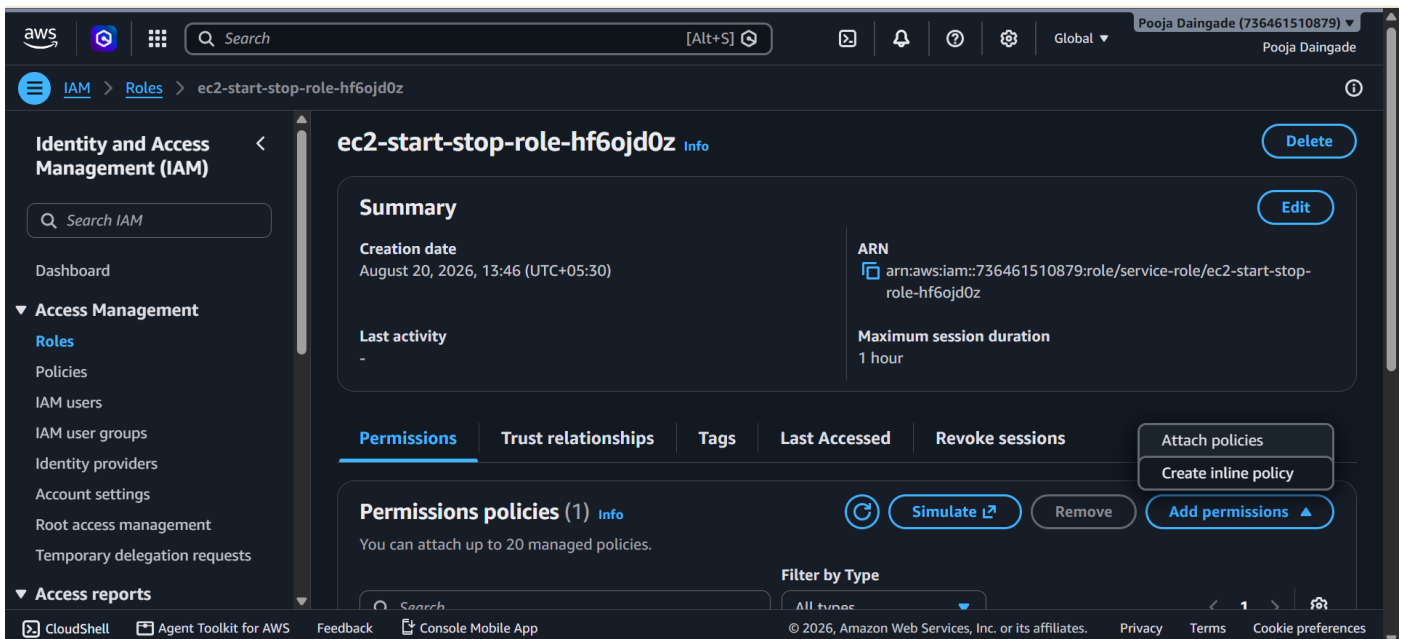
6. Configured the Lambda function with the required Python code for EC2 start/stop automation. The complete source code is available in the GitHub repository. Deployed the updated code by clicking the **Deploy** button.



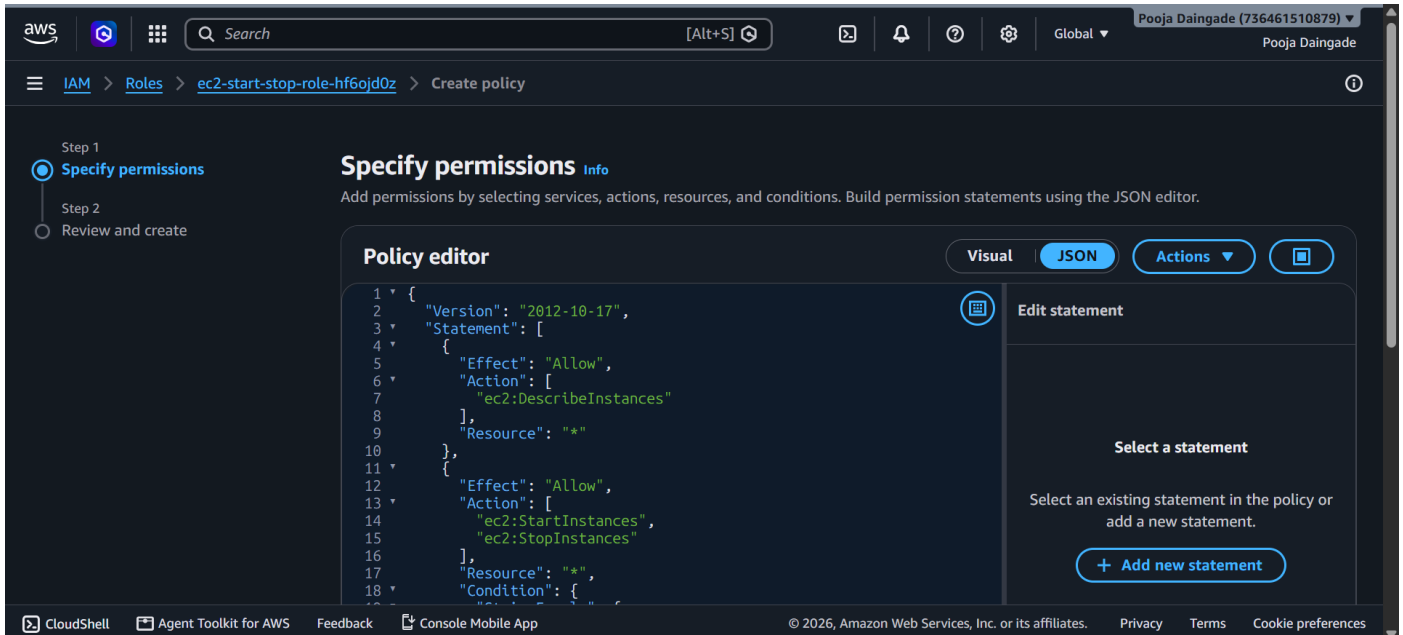
7. Navigate to the Lambda Function → **Configuration** → **Permissions**, then open the **EC2 start-stop execution role**.



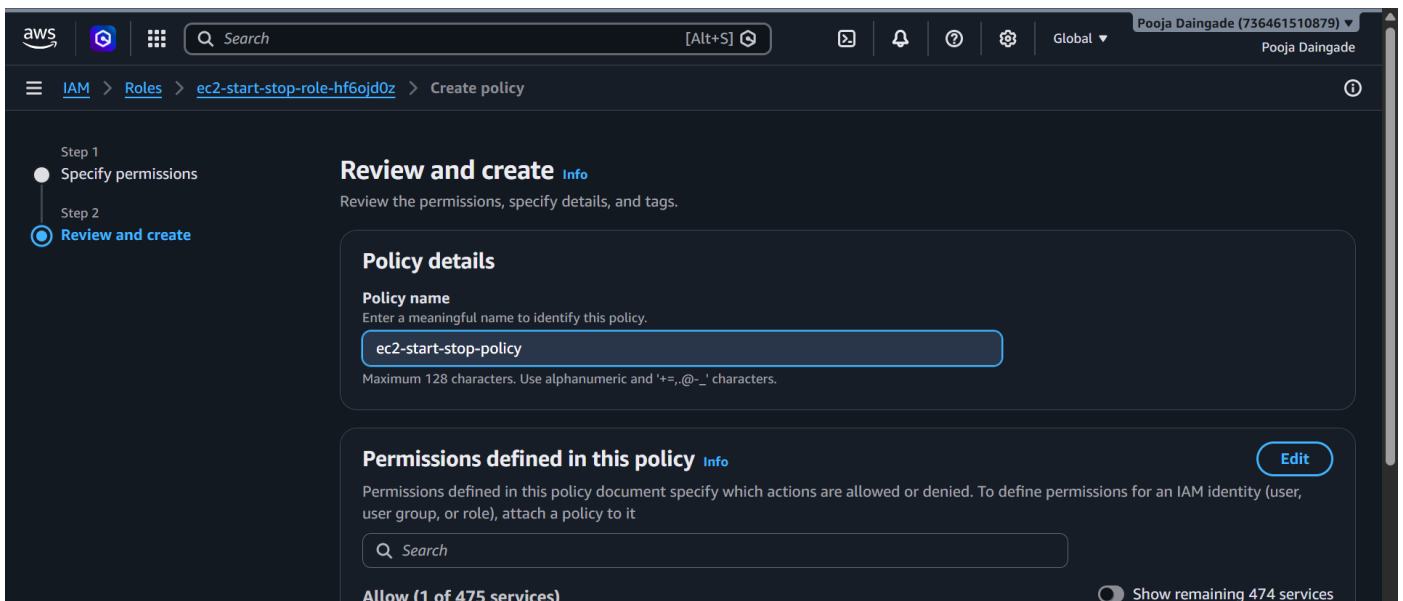
8. Opened the Lambda execution role, selected **Add permissions**, and chose **Create inline policy** to define the required EC2 permissions for the Lambda function.



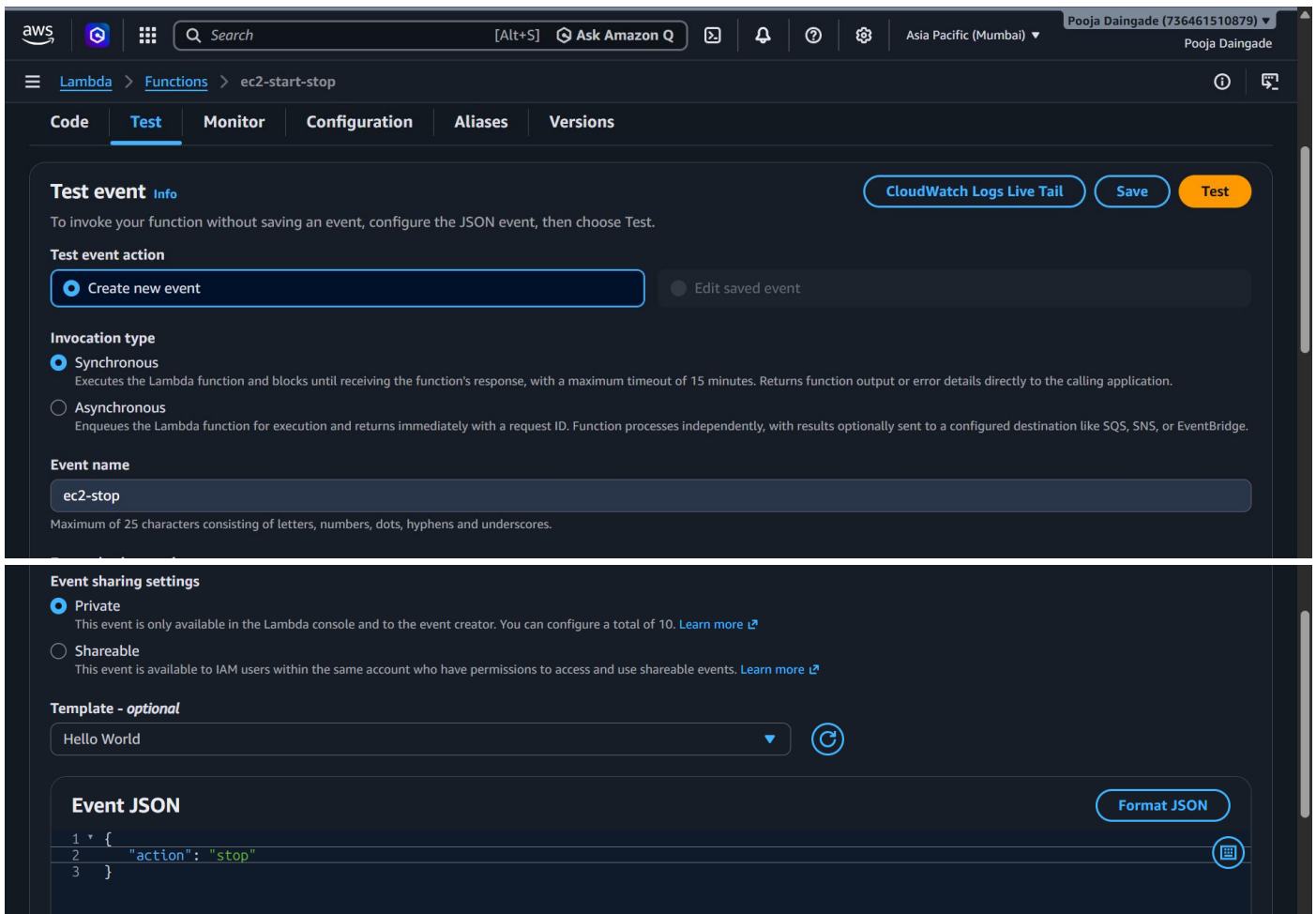
9. Selected the **JSON** editor, added the required IAM policy.
Complete policy code is available in the GitHub repository.



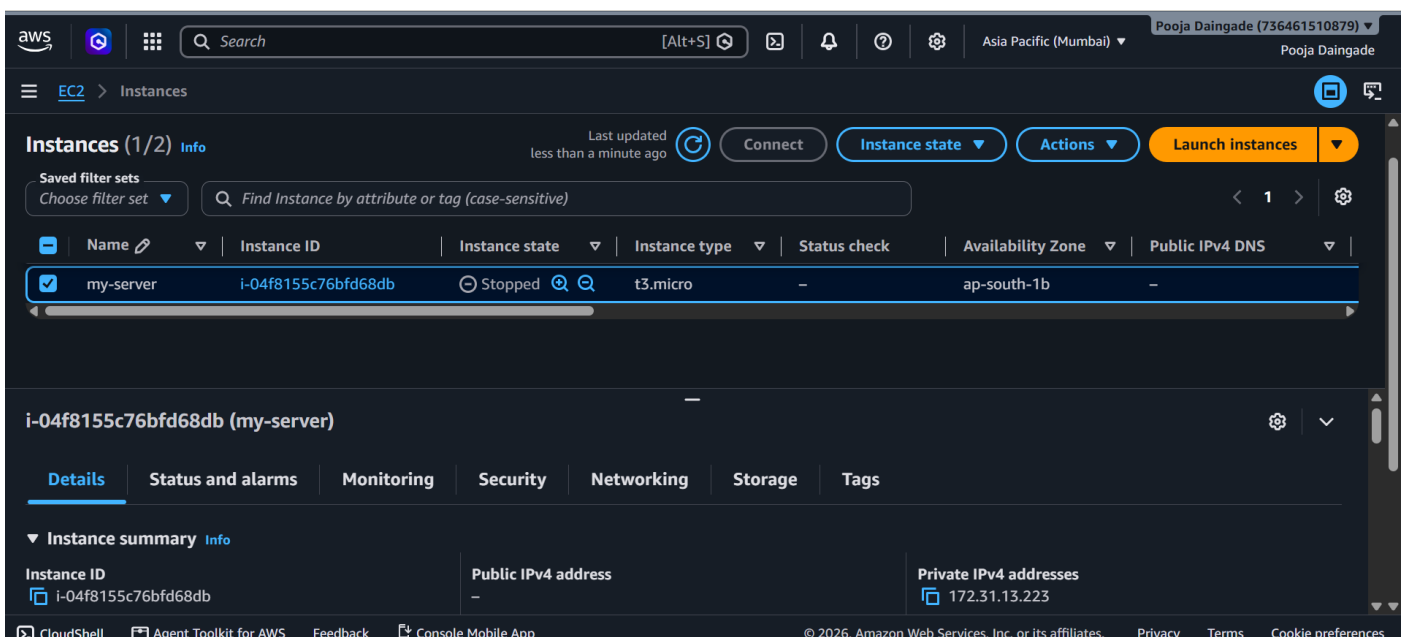
10. Entered the policy name **ec2-start-stop-policy** and clicked **Create policy** to attach the inline policy to the Lambda execution role.



11. Created a new Lambda test event named **ec2-stop** and configured the event JSON with **"action": "stop"**
Clicked Save and test.



12. Verified that the EC2 instance was successfully stopped after executing the ec2-stop Lambda test event.



13. Created an Amazon EventBridge schedule named **ec2-start**, selected the **One-time schedule** option, configured the required **date and time**, off the Flexible Time Window, and clicked **Next** to continue the schedule configuration.

The screenshot shows the 'Specify schedule detail' step of the Amazon EventBridge 'Create schedule' wizard. The left sidebar indicates the current step is 'Specify schedule detail' (Step 1), with other steps being 'Select target', 'Settings', and 'Review and create schedule'. The main content area is titled 'Specify schedule detail' and contains the following sections:

- Schedule name and description:**
 - Schedule name:** A text input field containing 'ec2-start'.
 - Description - optional:** A text area with the placeholder 'Enter description'.
 - Schedule group:** A dropdown menu set to 'default'.

At the bottom of the page, there are links for 'CloudShell', 'Agent Toolkit for AWS', 'Feedback', and 'Console Mobile App', along with copyright information and links for 'Privacy', 'Terms', and 'Cookie preferences'.

The second screenshot shows the 'Schedule pattern' step of the same wizard. The left sidebar indicates the current step is 'Select target' (Step 2). The main content area is titled 'Schedule pattern' and contains the following sections:

- Schedule pattern:**
 - Occurrence:** Two radio buttons: 'One-time schedule' (selected) and 'Recurring schedule'.
 - Date and time:**
 - Date:** A date picker showing '2026/08/20'.
 - Time:** A time input field showing '14:00'.
 - Time zone:** A dropdown menu showing '(UTC+05:30) Asia/Calcutta'.
 - Flexible time window:** A dropdown menu set to 'Off'.

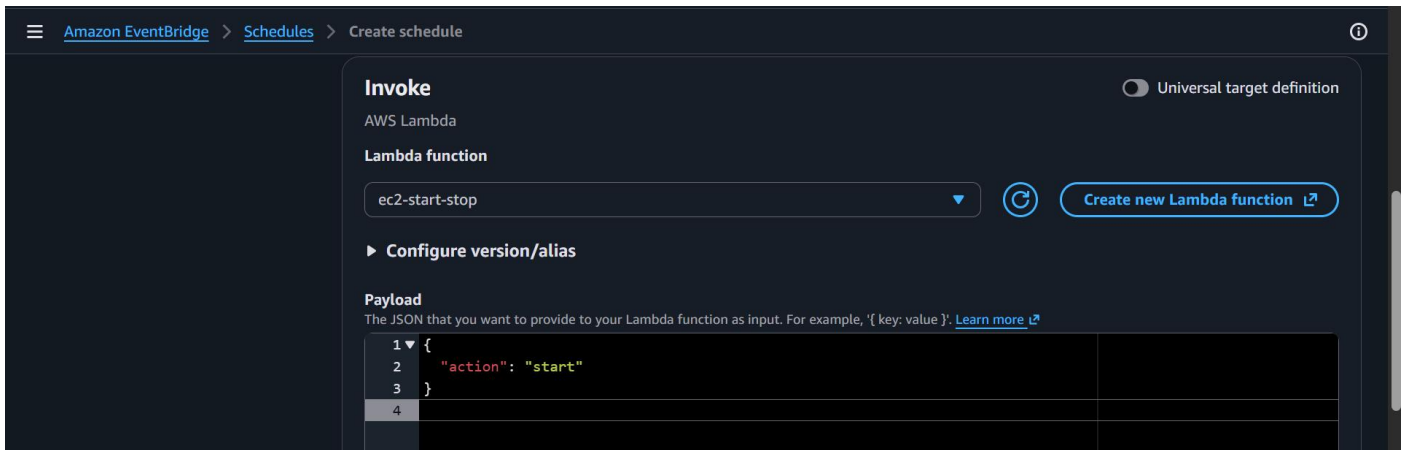
At the bottom right of the page, there are 'Cancel' and 'Next' buttons.

14. Selected **AWS Lambda Invoke** as the EventBridge target.

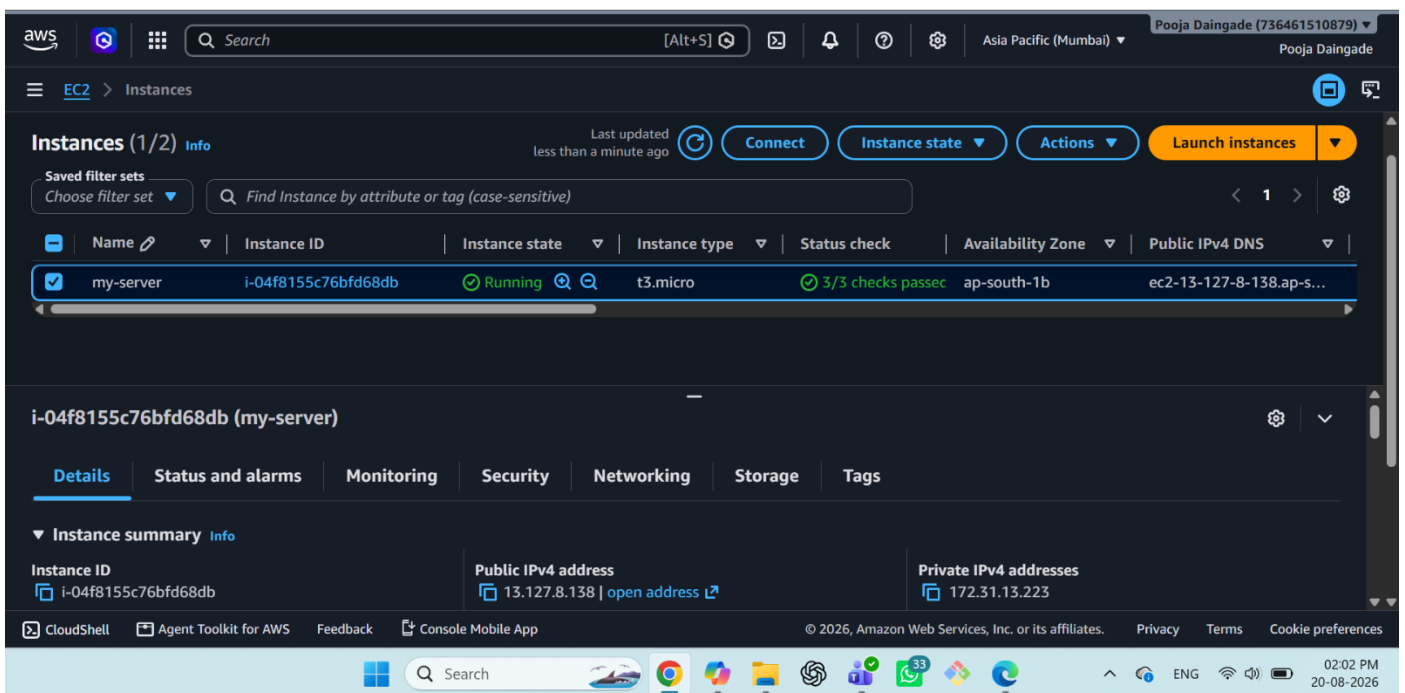
The screenshot shows the 'Select target' step of the Amazon EventBridge 'Create schedule' wizard. The left sidebar indicates the current step is 'Select target' (Step 2), with other steps being 'Specify schedule detail', 'Settings', and 'Review and create schedule'. The main content area is titled 'Select target' and contains the following sections:

- Target detail:**
 - Target API:** Two radio buttons: 'Templated targets' (selected) and 'All APIs'.
 - Templated targets:** A grid of target options, each with an icon and a radio button:
 - CodeBuild StartBuild
 - CodePipeline StartPipeline...
 - Amazon ECS RunTask
 - Amazon EventBridge PutEvents
 - Amazon Inspector V1 StartAssessm...
 - Kinesis Data Firehose PutRecord
 - Kinesis Data Streams PutRecord
 - AWS Lambda Invoke** (selected)
 - Amazon SNS
 - Amazon SQS
 - SageMaker
 - AWS Step Functions

15. Selected the **ec2-start-stop** Lambda function as the target, configured the payload with **"action": "start"**, and clicked **Create schedule**.



16. Verified that the EC2 instance was successfully started at the scheduled time.



Conclusion

The project successfully implements a secure, serverless, and cost-efficient EC2 scheduling solution using AWS Lambda and EventBridge Scheduler. By leveraging IAM least-privilege access and tag-based dynamic targeting, the solution enables automated EC2 lifecycle management, reduces unnecessary resource usage, and promotes efficient cloud infrastructure management.